

Performance evaluation of parallel programs

Pacheco, Section 2.6

Moreno Marzolla
Dip. di Informatica—Scienza e Ingegneria (DISI)
Università di Bologna

<https://www.moreno.marzolla.name/>

Copyright © 2013, 2014, 2017–2020, 2024, 2025
Moreno Marzolla, Università di Bologna, Italy
<https://www.moreno.marzolla.name/teaching/APAI/>



This work is licensed under the Creative Commons Attribution-ShareAlike 4.0 International License (CC-BY-SA 4.0). To view a copy of this license, visit <https://creativecommons.org/licenses/by-sa/4.0/> or send a letter to Creative Commons, 543 Howard Street, 5th Floor, San Francisco, California, 94105, USA.

Scalability

- How much faster can a given problem be solved with p workers instead of one? In a fixed resources (p) context
- How much more work can be done with p workers instead of one? In a fixed time context
- What impact for the communication requirements of the parallel application have on performance?
- What fraction of the resources is actually used productively for solving the problem? rather than communication and synchronization

Speedup

- Let us define:
 - p = Number of processors / cores for us processors ~ cores ~ threads
even if they are much different concepts
 - T_{serial} = Execution time of the serial program
 - $T_{\text{parallel}}(p)$ = Execution time of the parallel program with p processors / cores

In order to test the times we have to take the input as fixed and test then the execution time.

Speedup

- Speedup $S(p)$

$$S(p) = \frac{T_{\text{serial}}}{T_{\text{parallel}}(p)} \approx \frac{\overset{\text{Relative speedup}}{T_{\text{parallel}}(1)}}{T_{\text{parallel}}(p)}$$

You can compute it by only using the parallel version of the program and so you don't have to rewrite it

- In the ideal case, the parallel program requires $1/p$ the time of the sequential program
- $S(p) = p$ is the **ideal** case of **linear speedup**
 - Realistically, $S(p) \leq p$
 - Is it possible to observe $S(p) > p$?

We assume that ideally $S(p) = k/(1/kp) = p$

It is ideal as the more we increase p and the more synchronization is required and the slower the process becomes

Yes, we can observe super-linear speedup with heterogeneous hardware (?), better use of cache



Warning



- Never use a serial program to compute T_{serial}
 - If you do that, you might see a spurious superlinear speedup that is not there
- Always use the parallel program with $p = 1$ processors

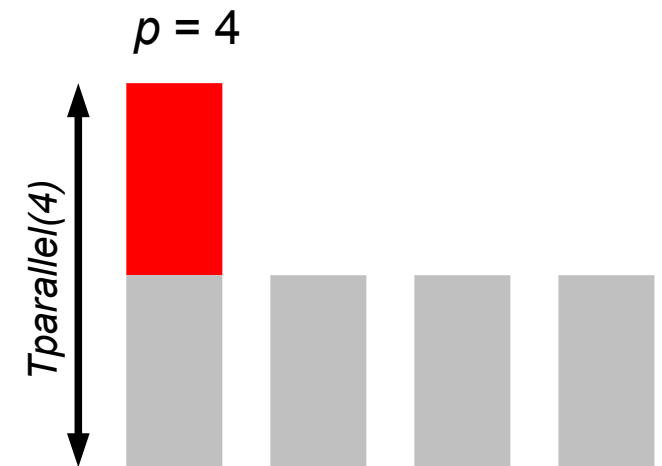
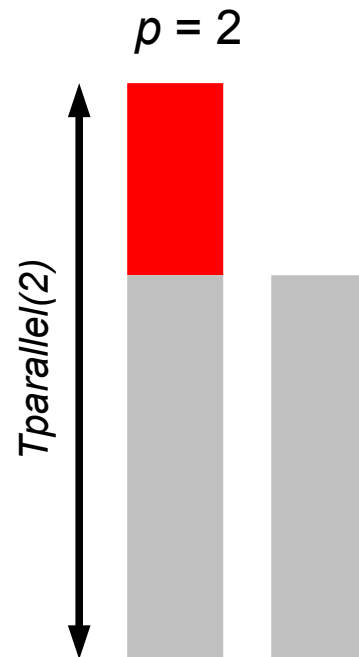
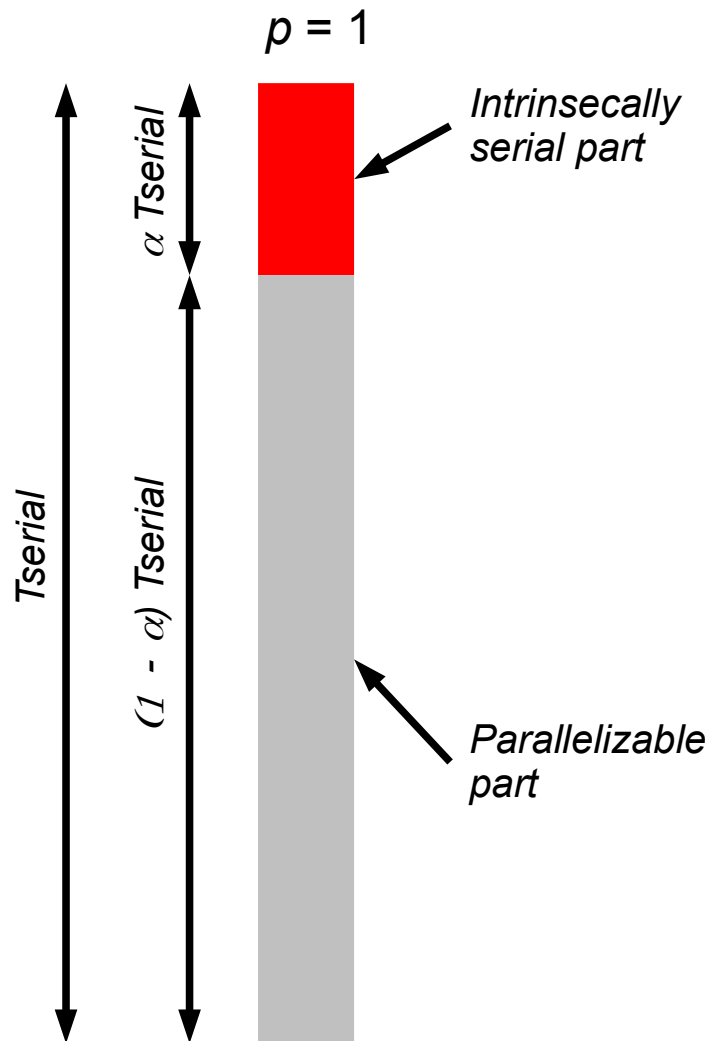
Non-parallelizable portions

alpha is in [0,1], where it is 1 when perfectly serial and 0 perfectly parallelizable

- Suppose that a fraction α of the total execution time of the serial program can **not** be parallelized
 - E.g., due to:
 - Algorithmic limitations (data dependencies)
 - Bottlenecks (e.g., shared resources)
 - Startup overhead
 - Communication costs
- Suppose that the remaining fraction $(1 - \alpha)$ can be **fully parallelized**
- Then, we have:

$$T_{\text{parallel}}(p) = \alpha T_{\text{serial}} + \frac{(1 - \alpha) T_{\text{serial}}}{p}$$

Example



$$T_{\text{parallel}}(p) = \alpha T_{\text{serial}} + \frac{(1 - \alpha) T_{\text{serial}}}{p}$$

Example

- Suppose that a program has $T_{\text{serial}} = 20\text{s}$
- Assume that 10% of the time is spent in a serial portion of the program (strong assumptions)
- Therefore, the execution time of a parallel version with p processors is

$$T_{\text{parallel}}(p) = 0.1 T_{\text{serial}} + \frac{0.9 T_{\text{serial}}}{p} = 2 + \frac{18}{p}$$

Example (cont.)

- The speedup is

$$S(p) = \frac{T_{\text{serial}}}{0.1 \times T_{\text{serial}} + \frac{0.9 \times T_{\text{serial}}}{p}} = \frac{20}{2 + \frac{18}{p}}$$

- What is the maximum speedup that can be achieved when $p \rightarrow +\infty$? $S(p \rightarrow \infty) = 1/(\alpha + (1-\alpha)/p) = 1/\alpha$

Amdahl's Law

- What is the maximum speedup?

$$\begin{aligned} S(p) &= \frac{T_{\text{serial}}}{T_{\text{parallel}}(p)} \\ &= \frac{T_{\text{serial}}}{\alpha T_{\text{serial}} + \frac{(1-\alpha) T_{\text{serial}}}{p}} \\ &= \frac{1}{\alpha + \frac{1-\alpha}{p}} \end{aligned}$$



Gene Myron Amdahl (1922—)

Amdahl's Law

- From

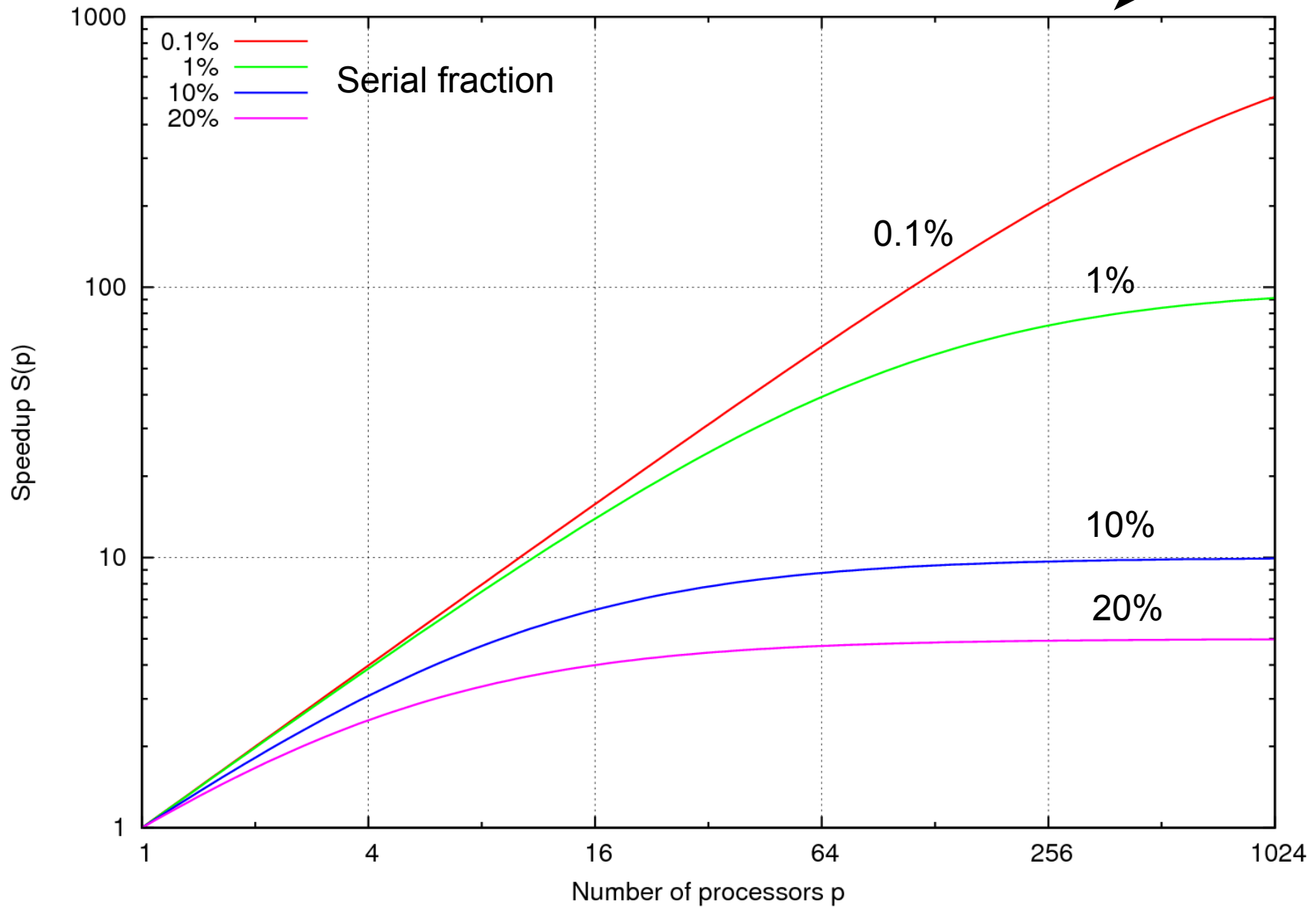
$$S(p) = \frac{1}{\alpha + \frac{(1-\alpha)}{p}}$$

we get an asymptotic speedup $1 / \alpha$ when p grows to infinity

If a fraction α of the total execution time is spent on a serial portion of the program, then the maximum achievable speedup is $1/\alpha$

Speedup

Note: log-log scale



Scaling Efficiency

- **Strong Scaling**: increase the number of processors p keeping the *total* problem size fixed
 - The *total* amount of work remains constant
 - The amount of work for a single processor decreases as p increases
 - Goal: reduce the total execution time by adding more processors
- **Weak Scaling**: increase the number of processors p keeping the *per-processor* work fixed
 - The total amount of work grows as p increases
 - The amount of work for a single processor remains the same as p increases
 - Goal: solve larger problems within the same amount of time

Strong Scaling Efficiency

- $E(p)$ = Strong Scaling Efficiency

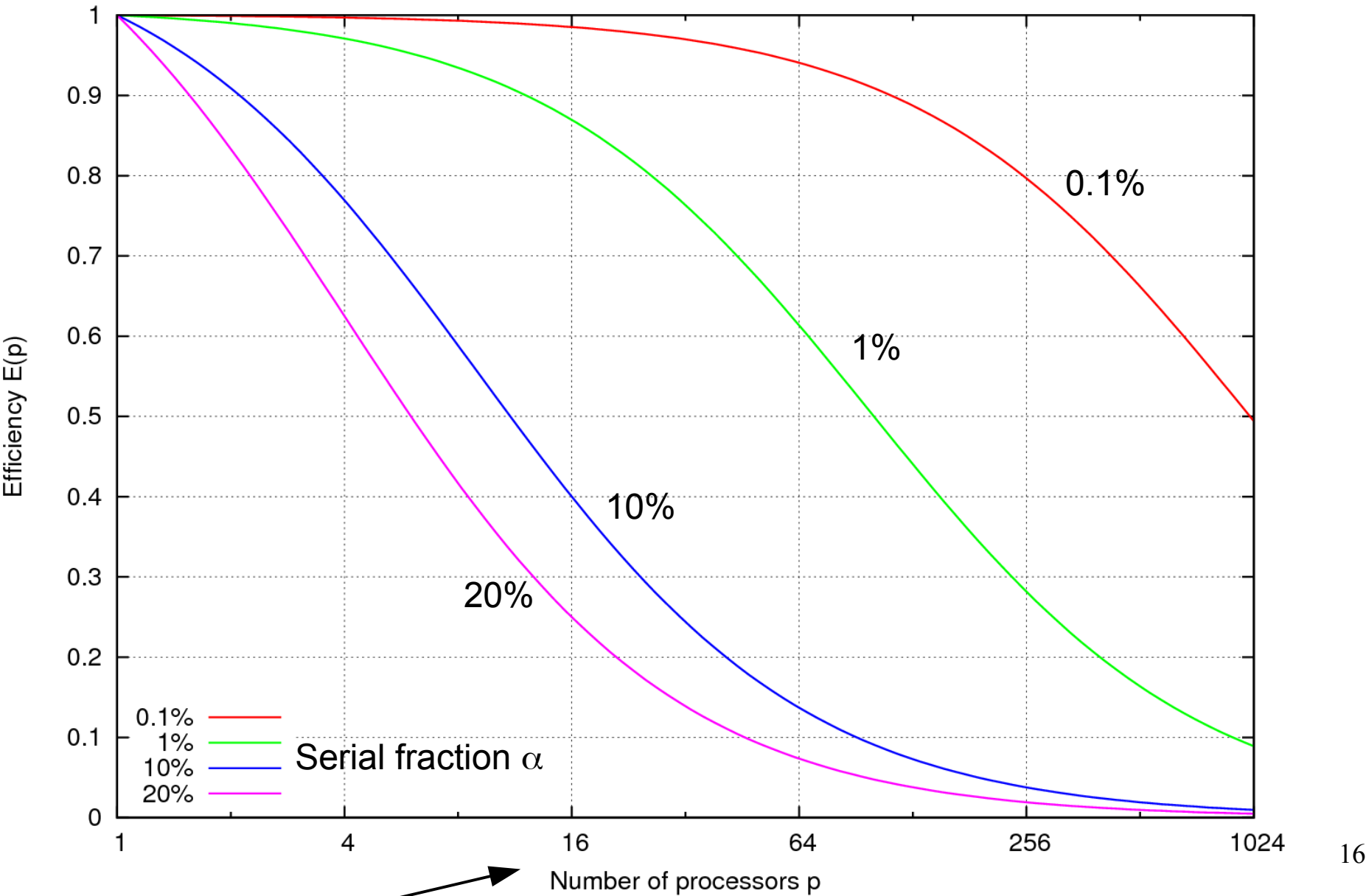
$$E(p) = \frac{S(p)}{p} = \frac{T_{\text{parallel}}(1)}{p \times T_{\text{parallel}}(p)}$$

it is in [0,1]

- where
 - $T_{\text{parallel}}(p)$ = Execution time of the parallel program with p processors / cores

HIGHER-IS-BETTER metric

Strong Scaling Efficiency and Amdahl's Law



Note: logarithmic x scale

Weak Scaling Efficiency

- $W(p)$ = Weak Scaling Efficiency

$$W(p) = \frac{T_1}{T_p}$$

- where
 - T_1 = time required to complete 1 work unit with 1 processor
 - T_p = time required to complete p work units with p processors

Before we could take the parallel program as a black box, now we have to know the internal behaviour as we have to divide the program into work units.

Weak scaling

The number of instructions

- Let $f(n_p, p)$ be the amount of work done by each processor, given input size n_p and p processors
- We want to compute the input size(s) n_p such that:

$$f(n_p, p) = \text{constant}$$

Example

Matrix-Matrix multiply

- For a given n_p , the amount of serial “work” required to compute the product of two $n_p \times n_p$ matrices is $O(n_p^3)$
- The OpenMP version using p processors performs $f(n_p, p) = n_p^3 / p$ work per OpenMP thread
- Therefore:

$$\begin{aligned} n_p^3 / p &= \text{const} \\ n_p &= \sqrt[3]{p \times \text{const}} \\ n_p &= \sqrt[3]{p} \times \text{const} ' \end{aligned}$$

This formula tells my how i have to increase the size of the input as we increase p when computing multiple tests for measuring $W(p)$. In this way each unit still computes the same amount of work and so the measurement is fair.

Example

- See [omp-matmul.c](#)
 - [demo-strong-scaling.sh](#) computes the times required for strong scaling (demo-strong-scaling.ods)
 - [demo-weak-scaling.sh](#) computes the times required for weak scaling (demo-weak-scaling.ods)

When doing scaling tests, in an hardware with 2 threads per core and 8 cores (16 parallel processes at most), we would notice that the speedup saturates around 8 instead of something closer to 16 (even lower than considering Amdahl's law) because once all 8 cores are fully occupied, adding the second thread per core only helps if there's idle functional capacity due to latency or memory stalls. In a well-balanced workload with few stalls, those extra threads fight for the same resources instead of exploiting unused ones, so performance gains flatten near the physical-core count rather than the logical-thread count.

The most precise method would be to count the clock cycles but as magnitudes increase it becomes impracticable and so we simply measure time which is also relatable from a customer point of view.

Taking times

- To compute speedup and efficiencies one needs to take the program **wall clock time**, excluding the input/output time

```
#if _XOPEN_SOURCE < 600  
#define _XOPEN_SOURCE 600  
#endif
```

```
#include "hpc.h"
```

```
other #includes here...
```

```
...
```

```
double start, finish;
```

```
/* read input data; this should not be measured */
```

```
start = hpc_gettime();
```

```
/* actual code that we want to time */
```

```
finish = hpc_gettime();
```

```
printf("Elapsed time = %e seconds\n", finish - start);
```

```
/* write output data; this should not be measured */
```

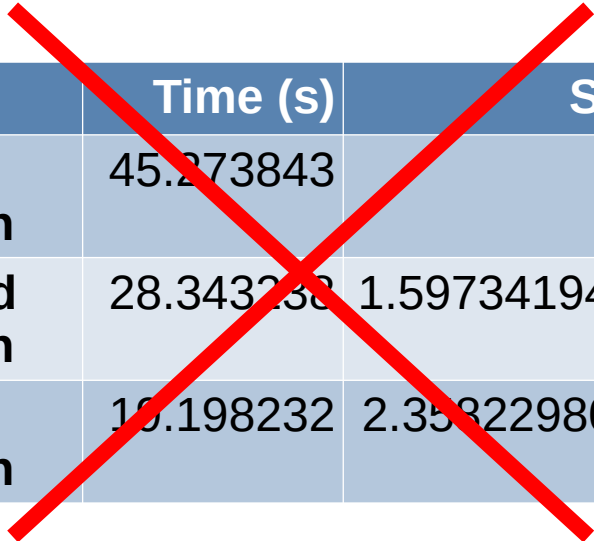
*Must be at the very beginning
(can be omitted for CUDA)*

*The actual
computation is done
here*

NOT INPUT/OUTPUT OPERATIONS

Displaying times

- When displaying times in seconds, e.g. in data tables, **do not show too many decimal digits** → We're not really capable of a so high precision
 - Rule of thumb: at most 3 decimal places (milliseconds)
 - Interference from other processes, the OS, I/O, ... introduce errors that are orders of magnitude higher



	Time (s)	Speedup
First version	45.273843	1
Second version	28.343238	1.59734194801596
Third version	19.198232	2.35822980991166

	Time (s)	Speedup
First version	45.274	1.00
Second version	28.343	1.60
Third version	19.198	2.36

Important

When showing performance results, always provide the **hardware specification**.

For the CPU

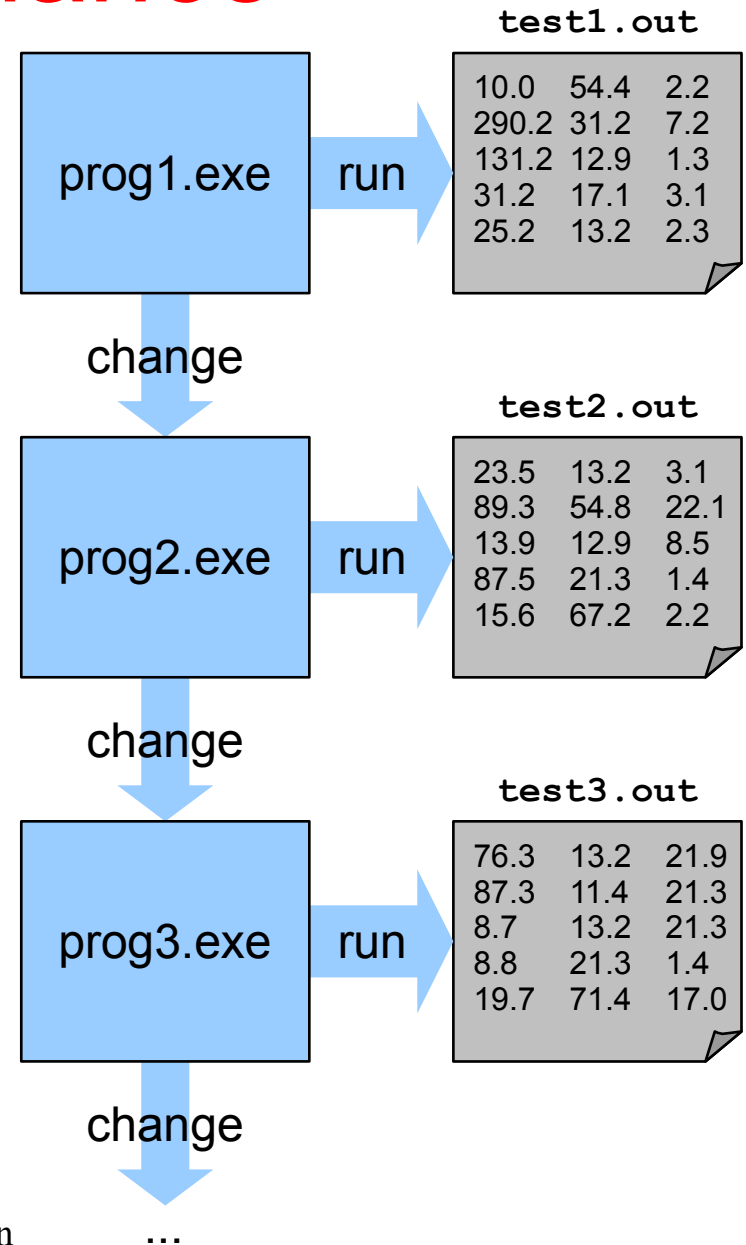
- Type of processor (vendor, model name, ecc.)
- Number of cores
- Whether the CPU uses symmetric multithreading (e.g., HyperThreading)
- Clock frequency
- Amount of RAM
- Operating System
- Compiler version and possibly compilation flags

For the GPU

- Type of GPU (vendor, model name, ecc.)
- Number of cores
- Clock frequency
- Amount of device memory
- Compiler version and possibly compilation flags

Data provenance

- You take a program
 - Run tests
 - Collect measures
- You change the program
 - Run tests
 - Collect measures
- You change the program *again*
 - Run tests
 - Collect measures
- ...



Data provenance

test1.out	test2.out	test3.out
10.0 54.4 2.2	23.5 13.2 3.1	76.3 13.2 21.9
290.2 31.2 7.2	89.3 54.8 22.1	87.3 11.4 21.3
131.2 12.9 1.3	13.9 12.9 8.5	8.7 13.2 21.3
31.2 17.1 3.1	87.5 21.3 1.4	8.8 21.3 1.4
25.2 13.2 2.3	15.6 67.2 2.2	19.7 71.4 17.0

- You end up with a bunch of output files
 - What do they mean?
 - How were the data collected?
- Make data **self-describing**
- Store metadata somewhere (e.g., in the output files) to document the conditions under which the tests were made:
 - The program name and command-line argument to the executable
 - The value of relevant environment variables
 - Compilation flags, if relevant
 - ... **use `lscpu` command in linux**

Example

```
# Demo strong scaling
# Test date: mar 14 ott 2025, 11:44:06, CEST
# host Linux 6.14.0-33-generic #33~24.04.1-Ubuntu SMP
PREEMPT_DYNAMIC Fri Sep 19 17:02:30 UTC 2 x86_64 x86_64 x86_64
GNU/Linux
# gcc (Ubuntu 13.3.0-6ubuntu2~24.04) 13.3.0
# Executable = ./omp-matmul
# matrix size = 2048
# max cores = 24
# n. of replications = 5
# OMP_SCHEDULE=static
# OMP_PLACES=
# OMP_PROC_BIND=
#  p    t1      t2      t3      t4      t5
#  1    13.715 13.701 13.719 13.694 13.643
#  2     7.104  7.103  7.107  7.100  7.095
#  3     4.749  4.749  4.749  4.749  4.815
#  4     3.636  3.617  3.635  3.635  3.554
#  5     2.912  2.902  2.902  2.910  2.909
#  6     2.439  2.647  2.638  2.626  2.631
#  ...
```

Post Scriptum: How to present results

Basic rules for graphing data

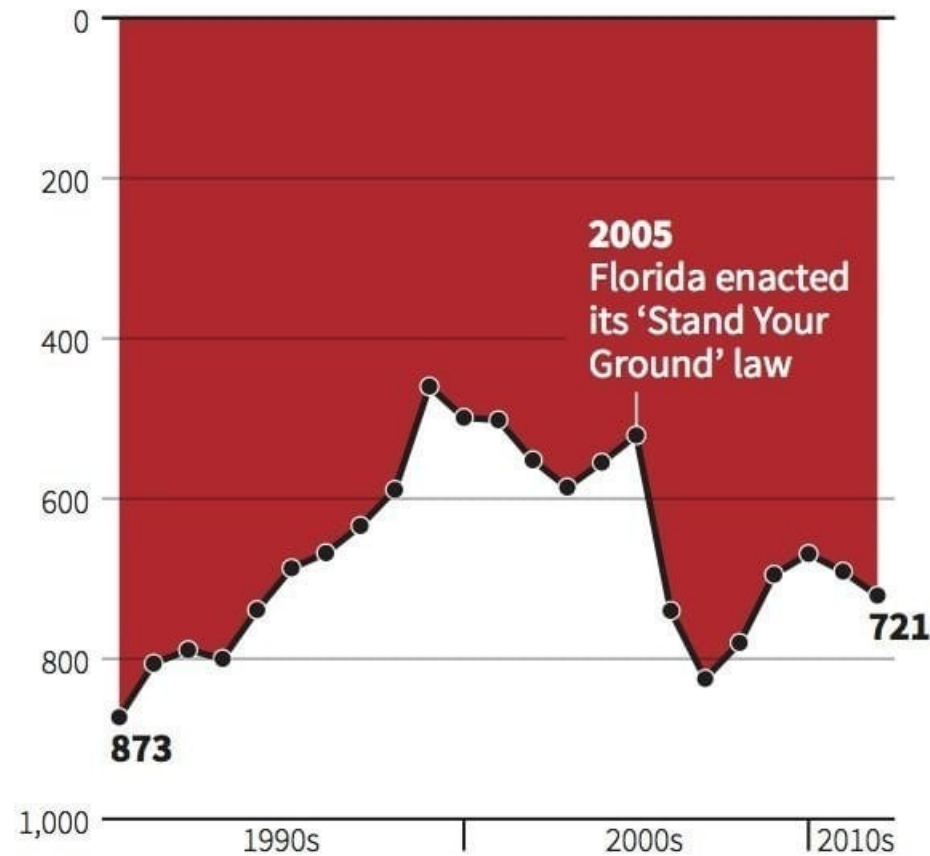
- Understanding the data should require the least possible effort from the reader
 - E.g., use names instead of symbols for the legend when possible
- Provide enough information to make the graph self-contained
- Stick to widely accepted practices
 - The origin of plots should be (0, 0)
 - The effect should be on the y axis, the cause on the x axis

Stick to accepted practices

Gun deaths in Florida

Number of murders committed using firearms

Bad
(vertical axis
grows
downwards)



Source: Florida Department of Law Enforcement

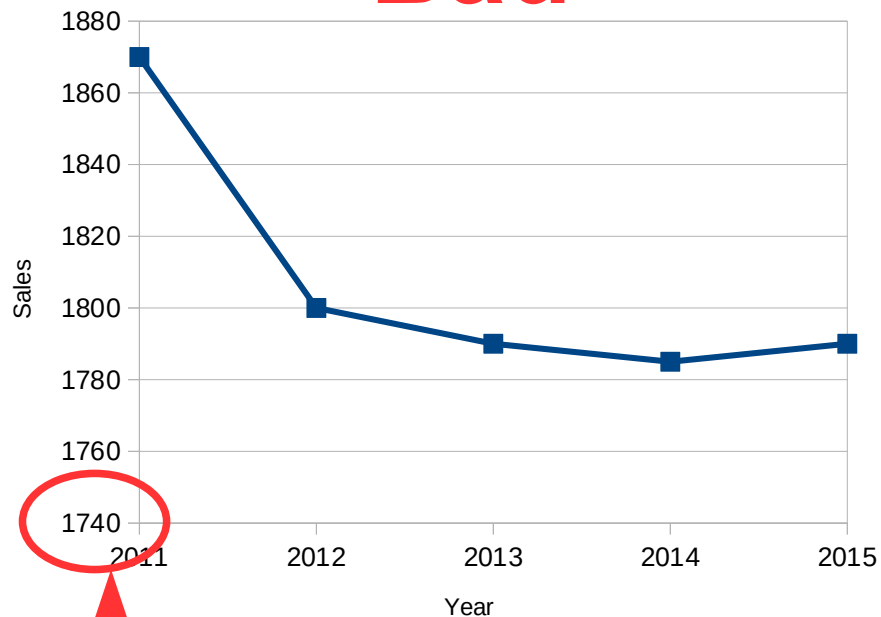
C. Chan 16/02/2014

REUTERS

Do not misuse axis origins

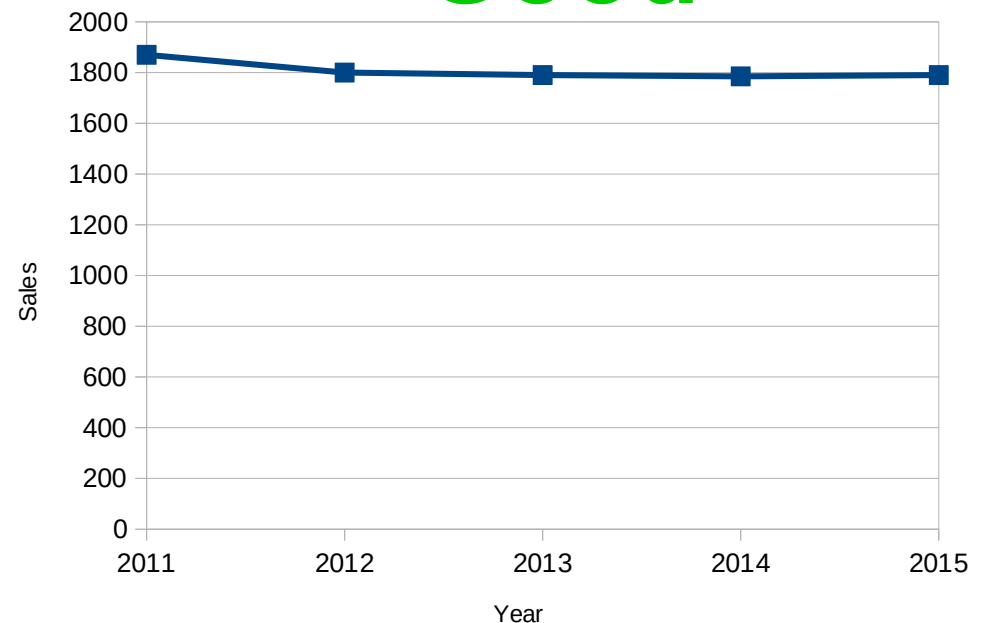
- Playing with the axis ranges allows one to emphasize small differences

Bad



Origin of vertical axis is not zero

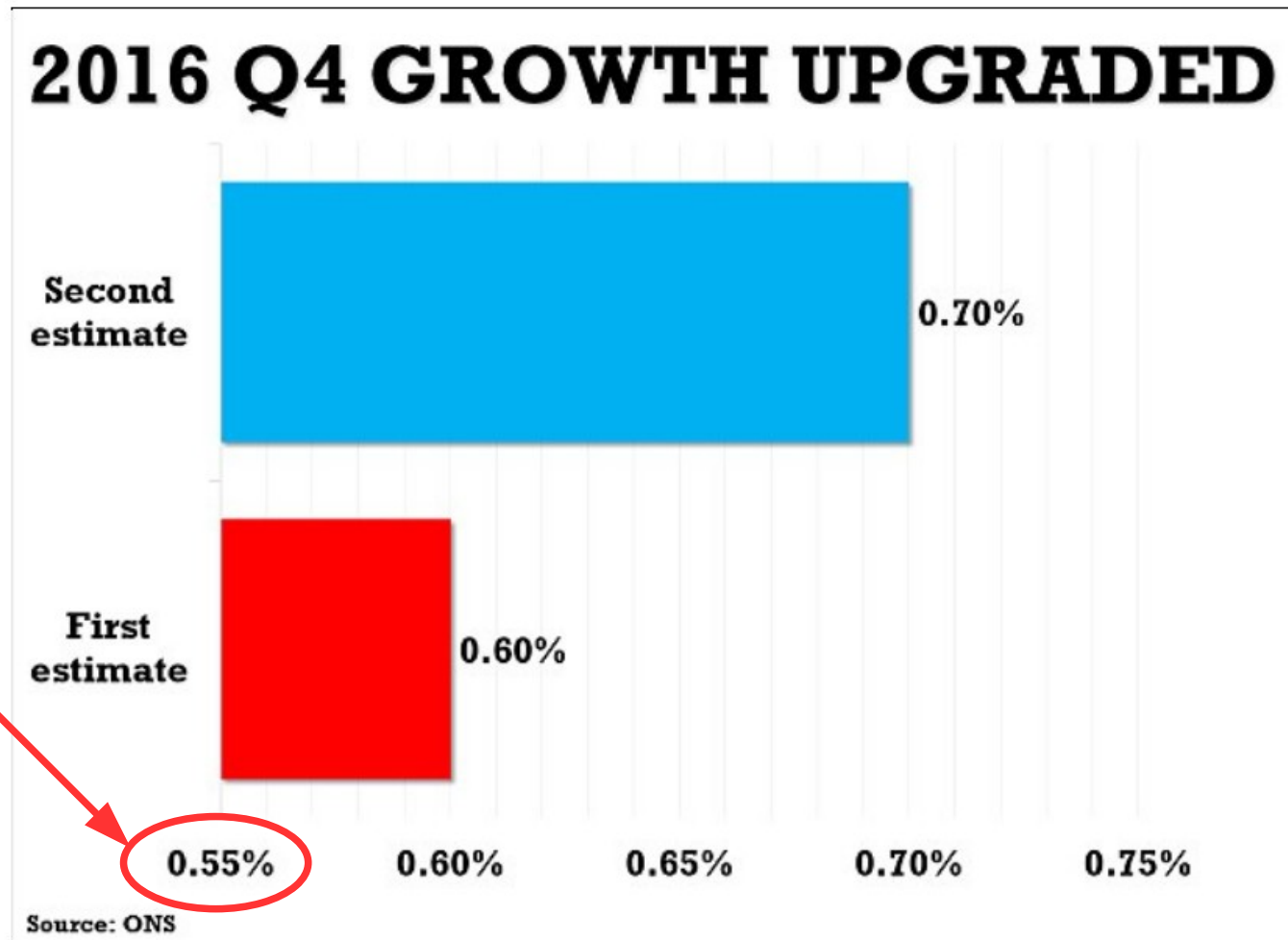
Good



Do not misuse axis origins

a change from 0.6% to 0.7% according to the Daily Mail

Bad
(origin is
not zero)



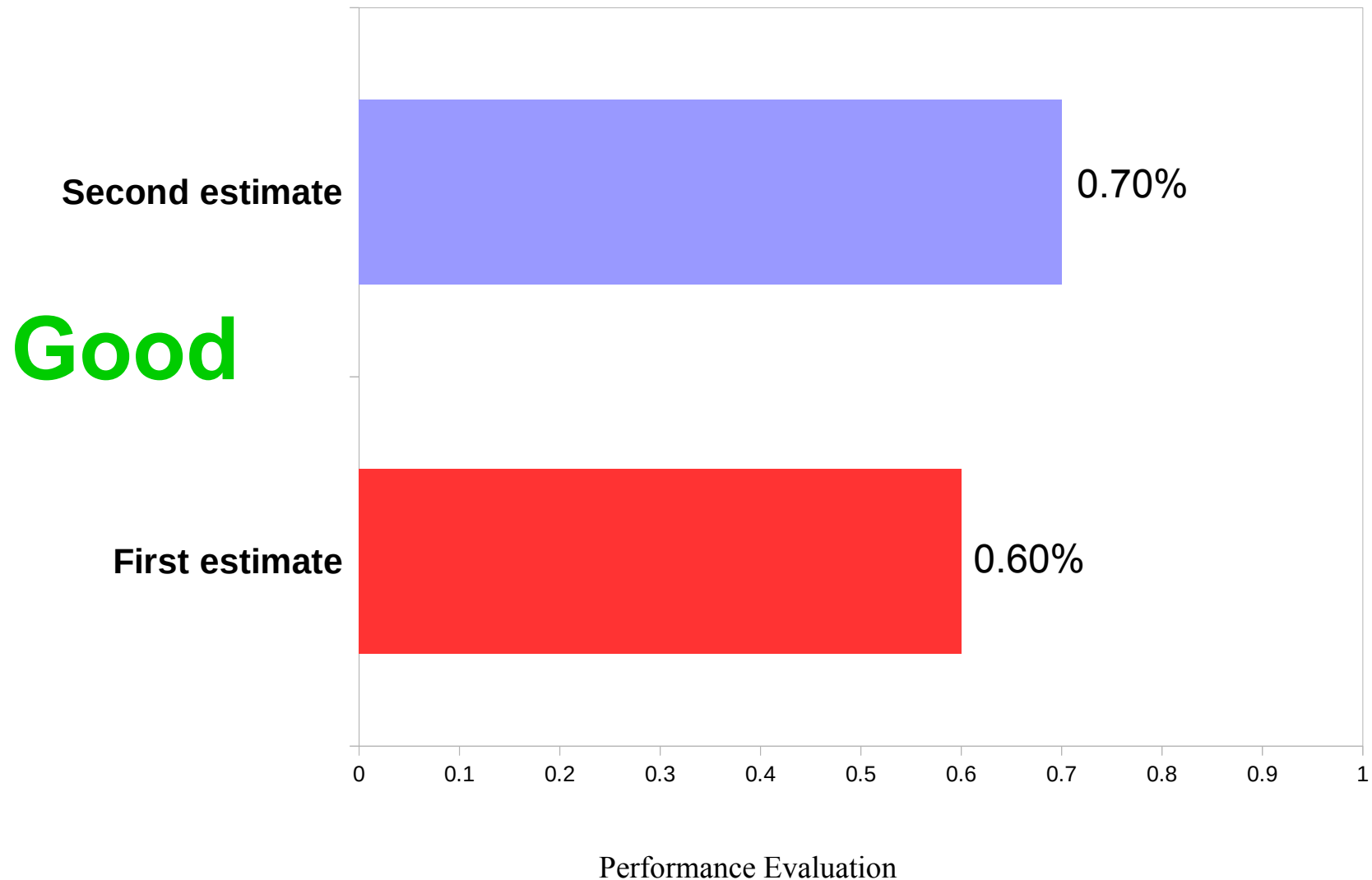
The Office for National Statistics (ONS) said gross domestic product (GDP) expanded by 0.7 per cent in the fourth quarter - an increase from the 0.6 per cent calculated on the watchdog's first look at the economy

<http://www.dailymail.co.uk/news/article-4248690/Economy-grew-0-7-final-three-months-2016.html>

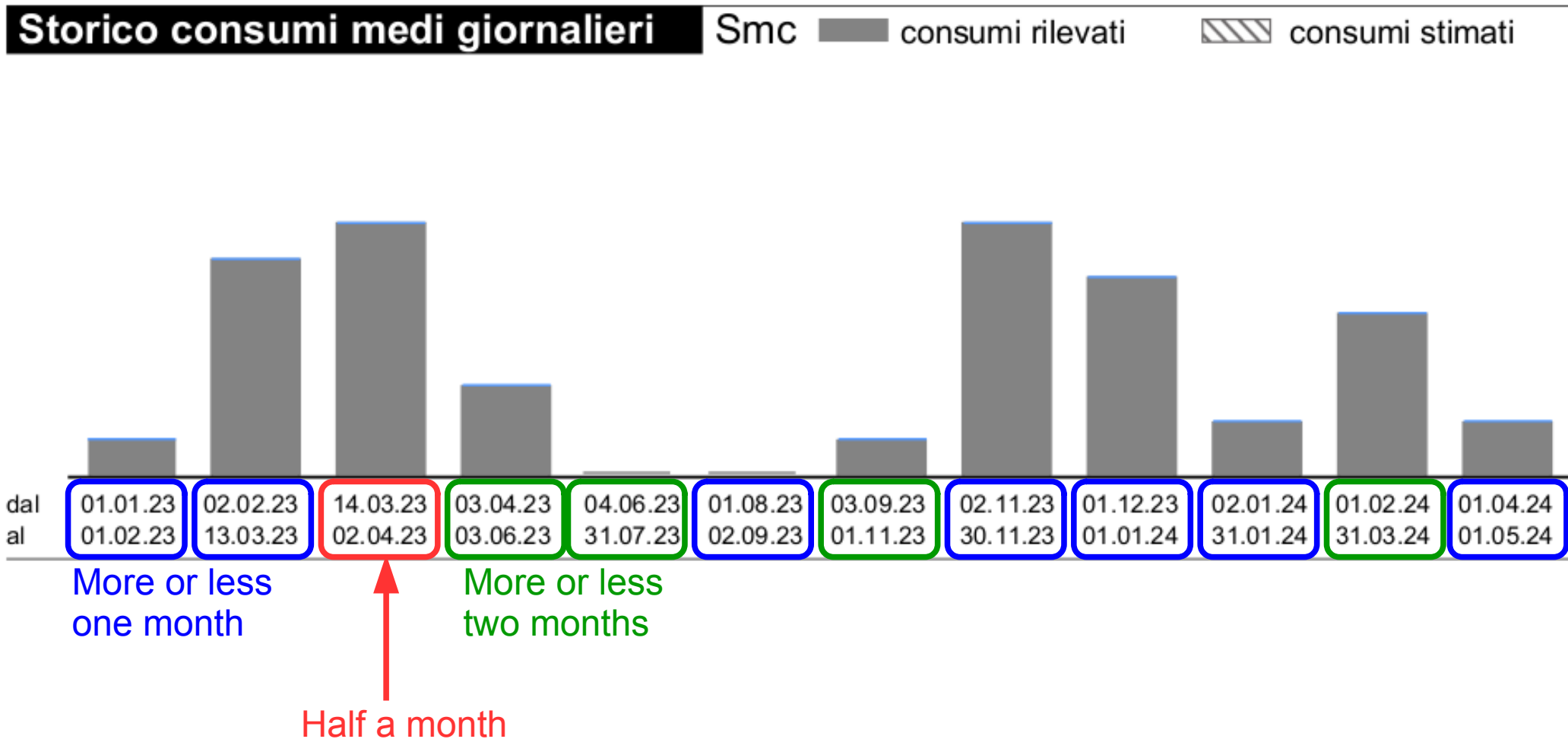
Do not misuse axis origins

a change from 0.6% to 0.7% with proper scale

2016 Q4 growth upgraded



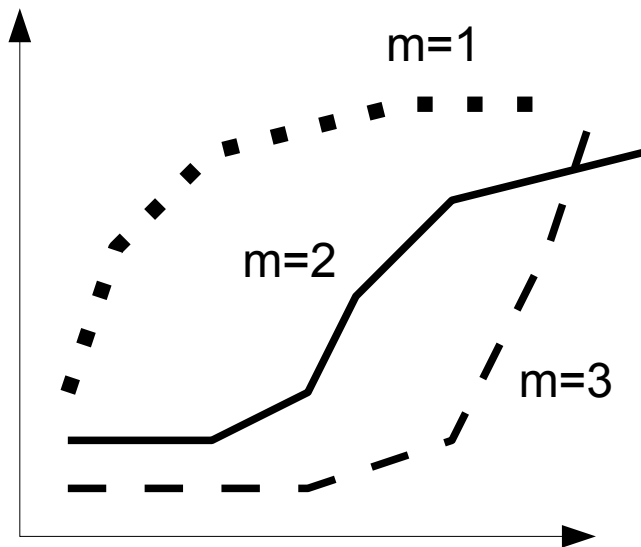
Do not misuse axis origins: Hera Edition



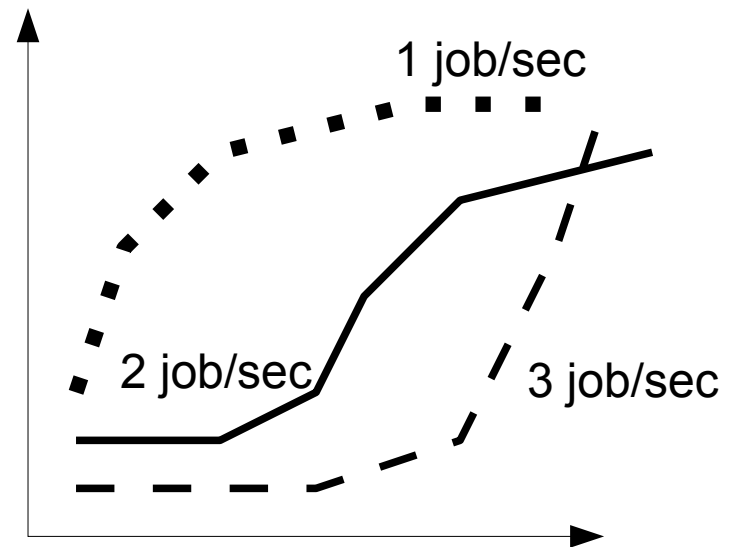
The unit on the x-axis is variable and not fixed

Prefer names to symbols

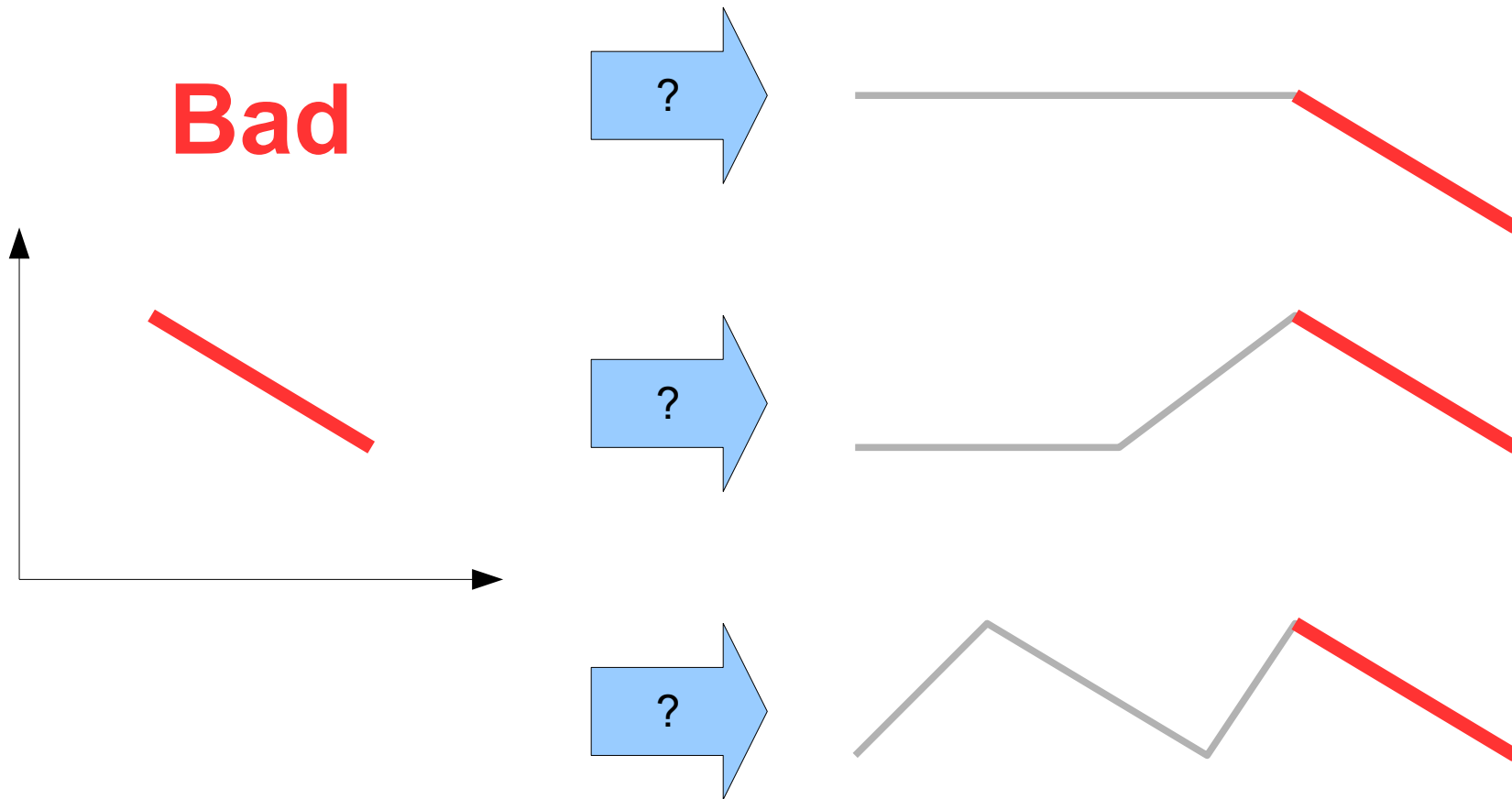
Bad



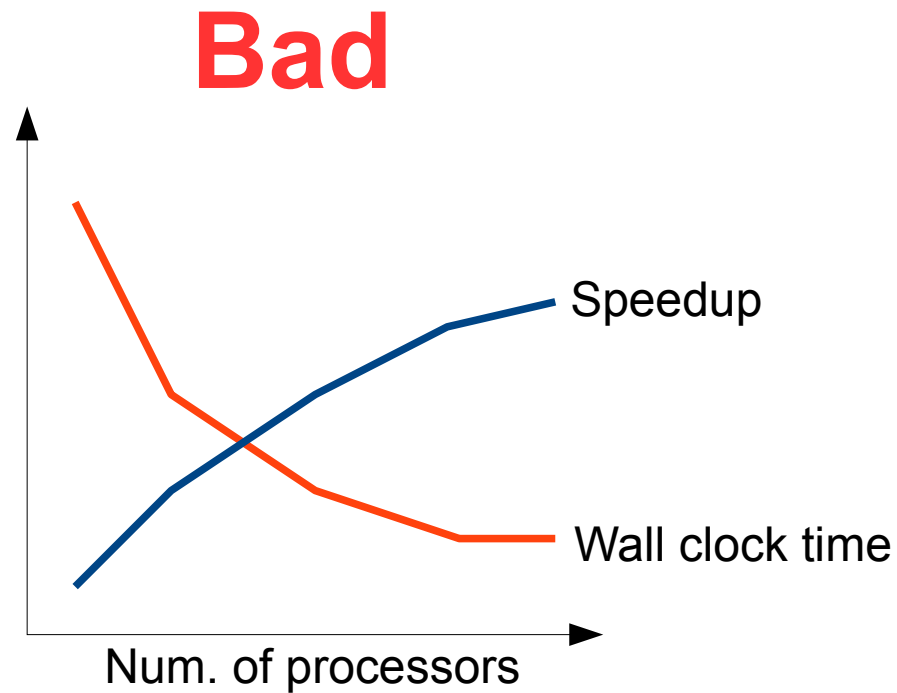
Good



Show all data



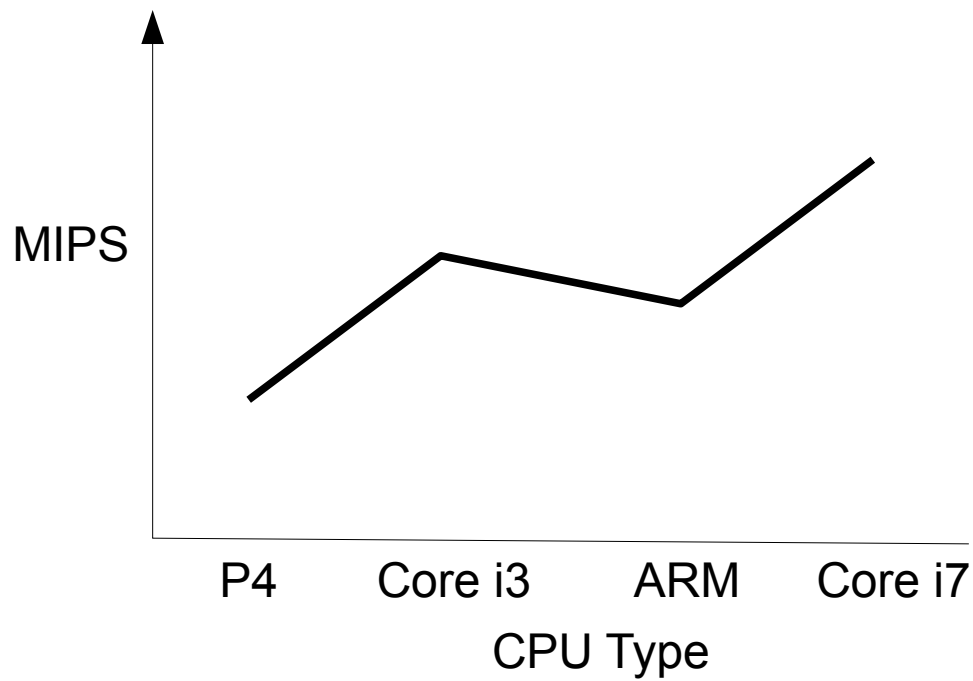
Do not overlap related data



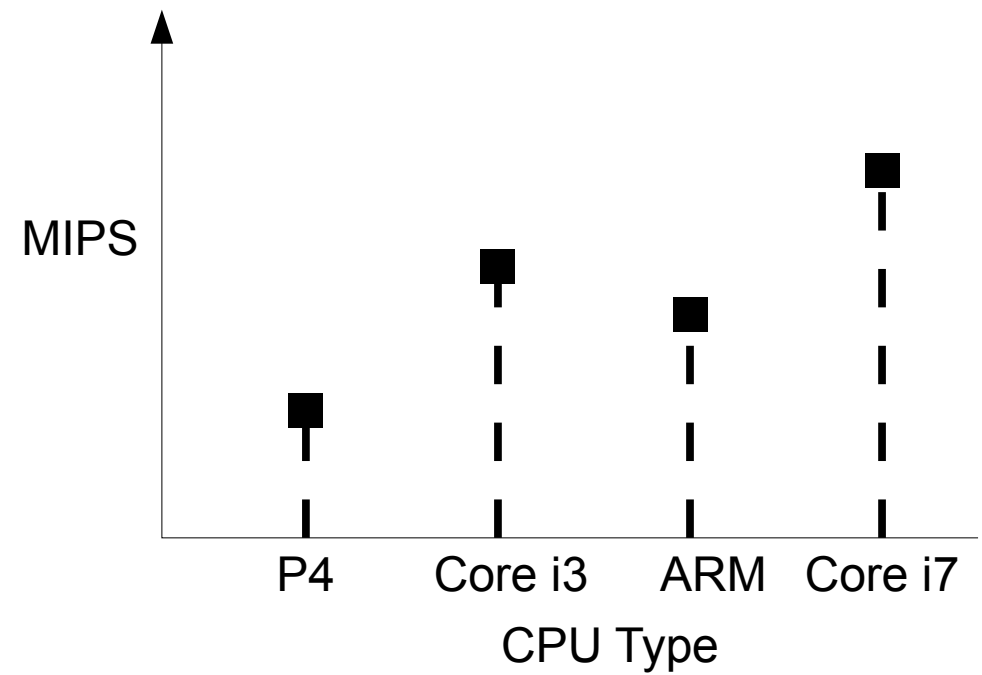
Do not connect data points of categorical variables

- **Categorical variables** assume values within a finite set of possible values that have no intrinsic ordering.

Bad

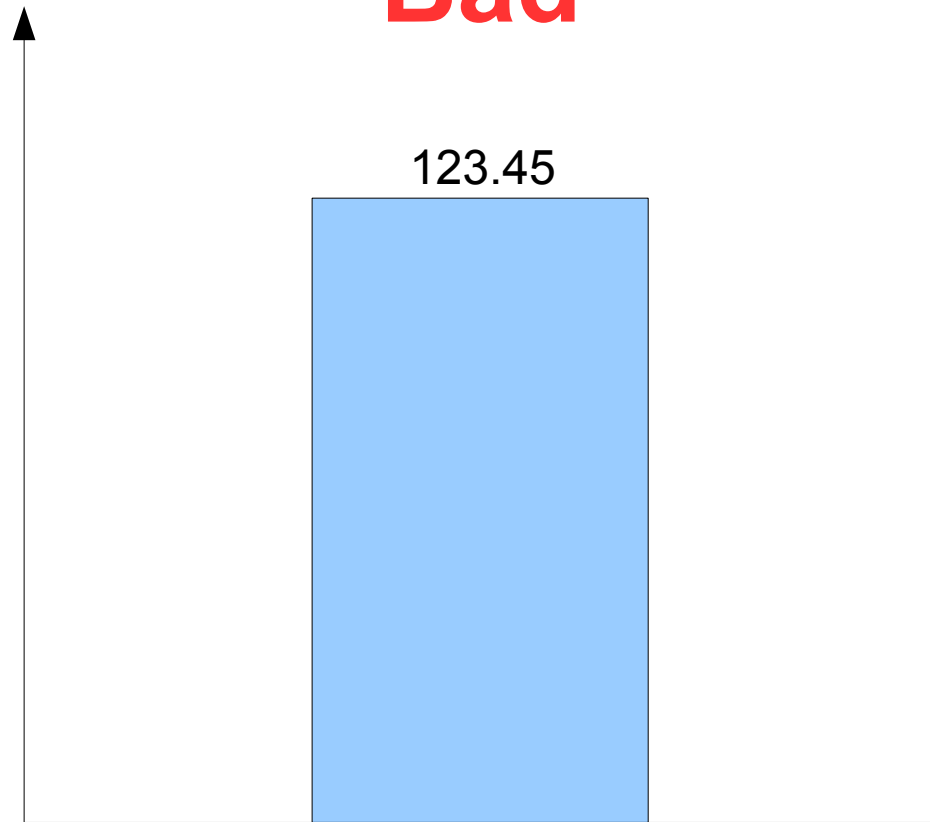


Better



Do not use graphs when words are enough

Bad



*I have actually seen
this in a thesis*

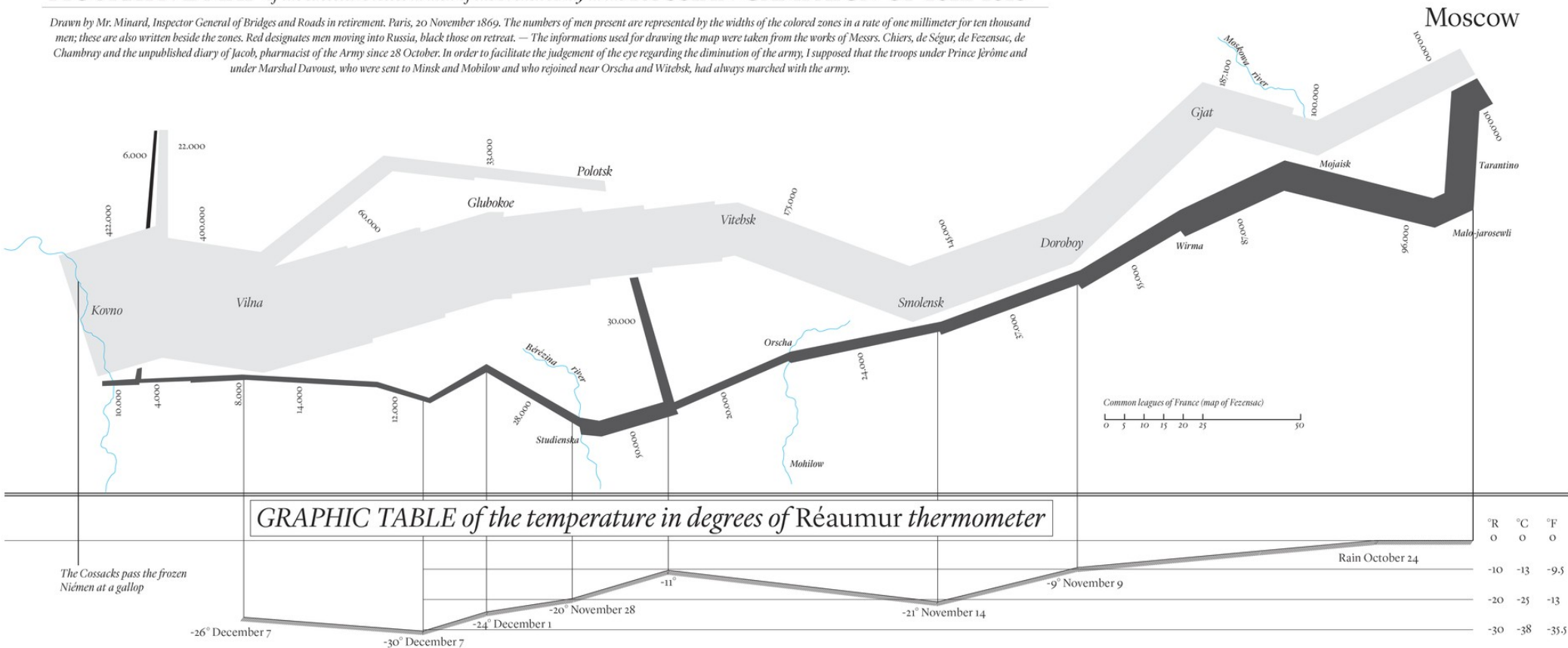
Other suggestions

- Always add a (numbered) caption to each figure
 - If possible, the caption should contain enough information to understand the figure without reading the main text
- All figures must be referenced and described in the main text of your document

“The best statistical graphic ever drawn” (Edward Tufte)

FIGURATIVE MAP of the successive losses in men of the French Army in the RUSSIAN CAMPAIGN OF 1812-1813

Drawn by Mr. Minard, Inspector General of Bridges and Roads in retirement, Paris, 20 November 1869. The numbers of men present are represented by the widths of the colored zones in a rate of one millimeter for ten thousand men; these are also written beside the zones. Red designates men moving into Russia, black those on retreat. — The informations used for drawing the map were taken from the works of Messrs. Chiers, de Ségur, de Fezensac, de Chambray and the unpublished diary of Jacob, pharmacist of the Army since 28 October. In order to facilitate the judgement of the eye regarding the diminution of the army, I supposed that the troops under Prince Jérôme and under Marshal Davoust, who were sent to Minsk and Mobilow and who rejoined near Orscha and Witebsk, had always marched with the army.



Modern redrawing of [Charles Minard](#)'s map of Napoleon's disastrous Russian campaign of 1812. The graphic is notable for its representation in two dimensions of six types of data: the number of Napoleon's troops; distance; temperature; the latitude and longitude; direction of travel; and location relative to specific dates (Image and description from Wikipedia)